Learning Objectives

Learners will be able to...

- **Define a queue and compare it to a stack**
- **Implement a queue with an array, a linked list, and a dynamic array**
- **Compare and contrast the different queue implementations**

Queue

What is a Queue?

A **queue** is a linear data structure that follows a particular order in which the operations are performed. Unlike stacks, where the order is last in, first out (LIFO), the order for queues is **first in, first out (FIFO)**. This means that the item that is inserted first is the first one to be removed. Imagine a real-life queue of people waiting in line for a bus; the first person to arrive is the first one to board.



Like stacks, queues are also an abstract data type. The ADT specifies which operations must be included (see below), but it does not state how the underlying data structure should be implemented. Whether you use an array, a linked list, or any other data structure to implement a queue, is entirely up to you and the requirements of your application.

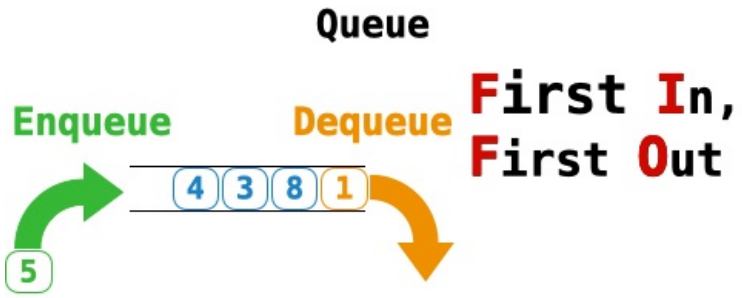
Queue Operations

A Queue supports a basic set of operations that make it unique:

- **Enqueue**: This operation adds an item to the back of the queue.
- **Dequeue**: This operation removes an item from the front of the queue.
- **Peek/Front**: This operation returns the item at the front of the queue without removing it.
- **IsEmpty**: Checks if the queue is empty.
- **IsFull**: Checks if the queue is full (applicable mainly to array-based implementations).

FIFO?

The FIFO paradigm is particularly useful in scenarios where fairness or order of processing is critical. Examples include job scheduling, order processing systems, and even data packet management in computer networks.



The rest of this assignment covers the different implementations of a queue and compares and contrasts these implementations.

Array-Based Implementation of Queue in C++

Representing a Queue with an Array

Now that we understand what a queue is and the operations it supports, let's implement one using an array. The choice of an array for this exercise offers simplicity and performs well for the operations we need.

In particular, we are going to create a circular queue. This means that once you fill up the array, you can "loop back around" and write over the elements that have already been dequeued.

Key Operations and Their Time Complexities

Here's a breakdown of the key operations and their time complexities for an array-based queue:

- **Enqueue**: $O(1)$
- **Dequeue**: $O(1)$
- **Peek/Front**: $O(1)$
- **IsEmpty**: $O(1)$
- **IsFull**: $O(1)$

Implementation

For our array-based Queue, we'll use a simple array with two pointers to keep track of the front and rear of the Queue. Since arrays are static, we also need to set the length of the array.

Create the `ArrayQueue` class with the attributes, `capacity`, `front`, `rear`, `size`, and `array`. `capacity` represents how many elements can be stored in the queue, `size` represents how many elements are currently in the queue, `front` points to the element at the front of the queue, `rear` points to the element at the rear of the queue, and `array` is the array used to store elements. In the constructor, set `capacity` to the size passed as a parameter, create an array with the specified size, set `front` to `0`, and set `rear` to `-1` since the array is empty.

```
class ArrayQueue {
private:
    int capacity; // Maximum capacity of the queue
    int front;    // Front index for dequeue operation
    int rear;     // Rear index for enqueue operation
```

```
    int size;    // Current size of the queue
    int* array;  // Array representing the queue

public:
    ArrayQueue(int c) {
        capacity = c;
        array = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    // Enqueue: Add an element to the rear of the queue
    void enqueue(int data) {
        if(isFull()) {
            throw std::runtime_error("Queue is full");
        }

        rear = (rear + 1) % capacity;
        array[rear] = data;
        size++;
    }

    // Dequeue: Remove an element from the front of the queue
    int dequeue() {
        if(isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }

        int data = array[front];
        front = (front + 1) % capacity;
        size--;
        return data;
    }

    // Peek: View the front element without removing it
    int peek() {
        if (isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }

        return array[front];
    }

    // Check if queue is empty
    bool isEmpty() {
        return size == 0;
    }

    // Check if queue is full
    bool isFull() {
        return size == capacity; // Circular queue condition for
        full
    }

    ~ArrayQueue() {
        delete[] array;
    }
```

```
};
```

In the `main` method, instantiate an array-based queue with a size of five elements. Add the integers `1`, `2`, and `3` to the queue. Peek at the first value and print it. Then dequeue the first element from the queue and print it. Finally, peek at the new first element and print it.

```
int main() {
    ArrayQueue myQueue(5);
    myQueue.enqueue(1);
    myQueue.enqueue(2);
    myQueue.enqueue(3);
    std::cout << "Peek: " << myQueue.peek() << std::endl;
    std::cout << "Dequeue: " << myQueue.dequeue() << std::endl;
    std::cout << "Peek: " << myQueue.peek() << std::endl;

    return 0;
}
```

You should see the following output:

```
Peek: 1
Dequeue: 1
Peek: 2
```

Linked List-Based Implementation of Queue

Representing a Queue with a Linked List

After discussing the array-based implementation of a queue, let's move on to using a linked list for the same purpose. This approach allows for dynamic resizing, offering a bit more flexibility compared to an array-based structure.

Key Operations and Their Time Complexities

Here's a breakdown of the key operations and their time complexities for a linked list-based Queue:

- **Enqueue**: $O(1)$
- **Dequeue**: $O(1)$
- **Peek/Front**: $O(1)$
- **IsEmpty**: $O(1)$

Implementation

In a linked list-based queue, we'll use a singly linked list with two pointers, `front` and `rear`, to keep track of the beginning and the end of the queue, respectively.

Start by creating the `Node` class. It has an attribute for the data stored and a pointer to the next node. In the constructor, set the `data` attribute to the value passed as a parameter, and set the pointer to `nullptr`.

```
// Define Node class for linked list elements
class Node {
public:
    int data;
    Node* next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

Then create the `LinkedListQueue` class. It has two attributes, `front` and `rear` which are of type `Node*`. In the constructor, set both of these attributes to `nullptr`.

```
class LinkedListQueue {
private:
    Node* front;  // Front of the queue
    Node* rear;   // Rear of the queue

public:
    // Initialize an empty queue
    LinkedListQueue() {
        front = nullptr;
        rear = nullptr;
    }

    // Enqueue: Add an element to the end of the queue
    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (isEmpty()) {
            front = newNode;
            rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
    }

    // Dequeue: Remove an element from the front of the queue
    int dequeue() {
        if (isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }
        int temp = front->data;
        Node* tempNode = front;
        front = front->next;
        delete tempNode;
        if (isEmpty()) {
            rear = nullptr; // If queue becomes empty
        }
        return temp;
    }

    // Peek: View the front element without removing it
    int peek() {
        if (isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }
        return front->data;
    }

    // Check if the queue is empty
    bool isEmpty() {
        return front == nullptr;
    }
```

```
    ~LinkedListQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }
};
```

In the `main` method, instantiate a linked list-based queue. Add the integers `1`, `2`, and `3` to the queue. Peek at the first value and print it. Then dequeue the first element from the queue and print it. Finally, peek at the new first element and print it.

```
int main() {
    LinkedListQueue myQueue;
    myQueue.enqueue(1);
    myQueue.enqueue(2);
    myQueue.enqueue(3);
    std::cout << "Peek: " << myQueue.peek() << std::endl;
    std::cout << "Dequeue: " << myQueue.dequeue() << std::endl;
    std::cout << "Peek: " << myQueue.peek() << std::endl;

    return 0;
}
```

You should see the following output:

```
Peek: 1
Dequeue: 1
Peek: 2
```

In this example, we add three elements (1, 2, 3) to the Queue, then peek at the front element (1), and dequeue it. This confirms that our linked list-based Queue implementation works as expected.

Dynamic Array-Based Implementation

Representing a Queue with a Dynamic Array

On the previous pages, we looked at array and linked list-based implementations of a queue. Now, let's explore another approach that utilizes dynamic arrays (such as `std::vector` in C++) for implementation. This is particularly useful when the size of the queue can change dynamically and you don't want to manage pointers yourself.

Key Operations and Their Time Complexities

Using dynamic arrays like `std::vector` can be convenient for queue implementation. Here are the time complexities for key operations in this case:

- **Enqueue**: $O(1)$ (amortized)
- **Dequeue**: $O(1)$
- **Peek/Front**: $O(1)$
- **IsEmpty**: $O(1)$

▼ What does "amortized" mean?
Amortized time complexity is an averaged time per operation over a sequence of operations, giving a more comprehensive understanding of performance. For example, in a dynamic array-based queue, most enqueue operations will be $O(1)$, but occasionally the operation will be more expensive due to the resizing of the array. When you average out the cost of these occasional expensive operations over a large number of total operations, you get a constant time complexity on average, often noted as $O(1)$ "amortized".

Implementation

Start by including the vector library. Then create the class `DynamicArrayQueue` which has two attributes, a `std::vector<int>` and an integer that keeps track of the front of the queue. In the constructor, instantiate the vector and set the front index pointer to `0`.

```
class DynamicArrayQueue {
private:
    std::vector<int> queue;
    int frontIndex;  // Keeps track of the front index

public:
    // Initialize an empty queue
    DynamicArrayQueue() {
        frontIndex = 0;
    }

    // Enqueue: Add an element to the end of the queue
    void enqueue(int value) {
        queue.push_back(value);  // std::vector handles resizing
    }

    // Dequeue: Remove an element from the front of the queue
    int dequeue() {
        if (isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }
        int temp = queue[frontIndex];
        frontIndex++;
        return temp;
    }

    // Peek: View the front element without removing it
    int peek() {
        if (isEmpty()) {
            throw std::runtime_error("Queue is empty");
        }
        return queue[frontIndex];
    }

    // Check if the queue is empty
    bool isEmpty() {
        return frontIndex >= queue.size();
    }
};
```

In the `main` method, instantiate a dynamic array-based queue. Add the integers `1`, `2`, and `3` to the queue. Peek at the first value and print it. Then dequeue the first element from the queue and print it. Finally, peek at the new first element and print it.

```
int main() {
    DynamicArrayQueue myQueue;
    myQueue.enqueue(1);
    myQueue.enqueue(2);
    myQueue.enqueue(3);
    std::cout << "Peek: " << myQueue.peek() << std::endl;
    std::cout << "Dequeue: " << myQueue.dequeue() << std::endl;
    std::cout << "Peek: " << myQueue.peek() << std::endl;

    return 0;
}
```

You should see the following output:

```
Peek: 1
Dequeue: 1
Peek: 2
```

In this example, we add three elements (1, 2, 3) to the Queue, then peek at the front element (1), and dequeue it. This confirms that our dynamic array-based Queue implementation works as expected.

Comparing Queue Implementations

Choosing the right data structure for implementing a queue is crucial for performance optimization. Below, we evaluate the three implementations discussed: array-based, linked list-based, and dynamic array-based queues, focusing on the pros and cons of each.

Array-Based Queue

Pros

- **Constant Time Operations**: Enqueue and Dequeue operations are $O(1)$ if no resizing is needed.
- **Memory Efficiency**: Uses a contiguous block of memory, making better use of caches.
- **Simplicity**: Easier to implement and understand.

Cons

- **Fixed Size**: The size needs to be defined upfront, which may lead to wasted space or the need for resizing.
- **Resizing Overhead**: If the array needs to be resized, it can be expensive.

Linked List-Based Queue

Pros

- **Dynamic Size**: The size can change dynamically, so there's no wasted space.
- **Constant Time Operations**: Both enqueue and dequeue operations are $O(1)$.

Cons

- **Memory Overhead**: Each node requires extra memory for the `next` pointer.
- **Cache Inefficiency**: Nodes might be scattered across the memory, leading to cache inefficiency.

Dynamic Array-Based Queue (vector)

Pros

- **Dynamic Resizing**: Automatically resizes, offering a balance between array and linked list implementations.
- **Amortized Constant Time**: Enqueue operation is $O(1)$ on average.

Cons

- **Amortized Time**: The time complexity is amortized, meaning occasionally operations may take longer.
- **Memory Overhead**: Internal resizing operations may temporarily require additional memory.

Comparison Table

Criteria	Array-Based	Linked List-Based	Dynamic Array-Based
Enqueue	$O(1)$	$O(1)$	$O(1)$ (amortized)
Dequeue	$O(1)$	$O(1)$	$O(1)$
Peek/Front	$O(1)$	$O(1)$	$O(1)$
Memory Efficiency	High	Moderate	Moderate
Cache Efficiency	High	Low	Moderate
Flexibility	Low	High	High

By understanding the strengths and weaknesses of each implementation, you can choose the one that best fits your specific needs.

Formative Assessment 1

Formative Assessment 2